

Programowanie Systemowe

Laboratorium: Pamięć współdzielona i semaforey

Jakub Regulski – 283866

Data wykonania ćwiczenia: 17.12.2025

Prowadzący: mgr inż. Przemysław Świercz

Kod:

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include "shared.h"

int main(int argc, char* argv[]) {
    if (argc != 2) {
        printf("Error: Zła liczba argumentow\n");
        return 1;
    }

    const char* shm_name = "/IPC";
    int fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, sizeof(struct shared_data));
    struct shared_data *data = mmap(NULL, sizeof(struct shared_data), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    if (strcmp(argv[1], "server") == 0) {
        server(data);
        shm_unlink(shm_name);
    } else if (strcmp(argv[1], "worker") == 0) {
        worker(data);
    }

    munmap(data, sizeof(struct shared_data));
    close(fd);
    return 0;
}
```

worker.c

```
#include <stdio.h>
#include <string.h>
#include "shared.h"

void worker(struct shared_data *data) {
    while (1) {
        sem_wait(&data->sem);

        if (strcmp(data->text, "EXIT") == 0) {
            sem_post(&data->sem);
            break;
        }

        data->text[0] = 'X';
        sem_post(&data->sem);
    }
}
```

server.c

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include "shared.h"

void server(struct shared_data *data) {
    sem_init(&data->sem, 1, 1);

    while (1) {

        sem_wait(&data->sem);
        printf("Podaj tekst: ");
        scanf("%s", data->text);
        if (strcmp(data->text, "EXIT") == 0) {
            sem_post(&data->sem);
            break;
        }

        sem_post(&data->sem);

        usleep(1000);

        sem_wait(&data->sem);
        printf("Przerobiony tekst: %s\n", data->text);
        sem_post(&data->sem);
    }
}
```

shared.h

```
#ifndef SHARED_H
#define SHARED_H

#include <semaphore.h>

struct shared_data {
    sem_t sem;
    char text[2048];
};

void server(struct shared_data *data);
void worker(struct shared_data *data);

#endif
```

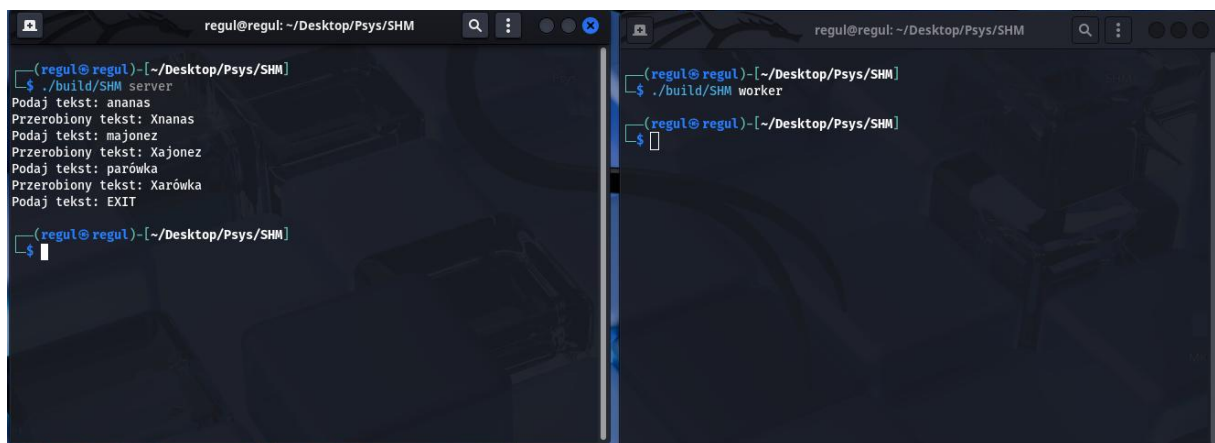
Wyjaśnienie działania:

Uruchamiamy program w dwóch osobnych terminalach, przekazując mu odpowiednie parametry.

1. Program uruchomiony z parametrem `server` pobiera od użytkownika tekst, a następnie zapisuje go do pamięci współdzielonej. Po zapisaniu danych oczekuje, aż program `worker` wykona swoją operację. Po zakończeniu pracy przez proces `worker`, program `server` odczytuje zmodyfikowany tekst z pamięci współdzielonej i wypisuje go.
2. Program uruchomiony z parametrem `worker` oczekuje na zapis danych do pamięci współdzielonej przez proces `server`. Po otrzymaniu sygnału synchronizacyjnego odczytuje tekst, dokonuje jego modyfikacji, a następnie zapisuje wynik z powrotem do pamięci współdzielonej.

W celu synchronizacji pracy procesów zastosowano jeden semafor. Ze względu na jego użycie, dla poprawnego działania programu wymagane jest uruchomienie procesu `server` jako pierwszego.

Wynik działania:



```
(regul@regul) ~/Desktop/Phys/SHM
$ ./build/SHM server
Podaj tekst: ananas
Przerobiony tekst: Xnanas
Podaj tekst: majonez
Przerobiony tekst: Xajonez
Podaj tekst: parówka
Przerobiony tekst: Xarówka
Podaj tekst: EXIT
$

(regul@regul) ~/Desktop/Phys/SHM
$ ./build/SHM worker
$
```